



Building a Virtual Hardware Device from Scratch using MMIO in QEMU

...



Ahmed Dajani

Published Feb 22, 2026

+ Follow

Introduction

Imagine you are working with a prototype device — a custom FPGA peripheral, an early silicon bring-up board, or a sensor module your hardware team just taped out. You need to start developing your Linux device driver, RTOS driver, or bare-metal application. But the hardware does not exist yet, or you only have one board shared across the whole team.

The traditional answer is "you wait." The better answer is: **emulate the hardware in software**. This is exactly what QEMU makes possible. You can model your device's registers and behavior in C, wire it into a full virtual machine running a real Linux

kernel, and write and test your driver against it — all before a single board ships. When the hardware arrives, you are porting, not starting from scratch.

What This Article Builds, Step by Step

Throughout this article we will develop a simple MMIO-based calculator device entirely in software. The calculator has six registers — two operands, an operation selector, a trigger, a result, and a status — and supports addition, subtraction, multiplication, and division. It is deliberately trivial by design. The goal is not the arithmetic; it is the complete workflow: modeling the hardware in QEMU, mapping it into the system bus, and writing userspace code that talks to it through **Memory-mapped I/O (MMIO)**.

What is MMIO? The x86 System Bus

In an x86 system, the CPU communicates with all peripherals through a shared address space on the system bus. RAM, ROM, and I/O devices all coexist in this space. When the CPU issues a memory read or write, the bus fabric decides which endpoint responds.

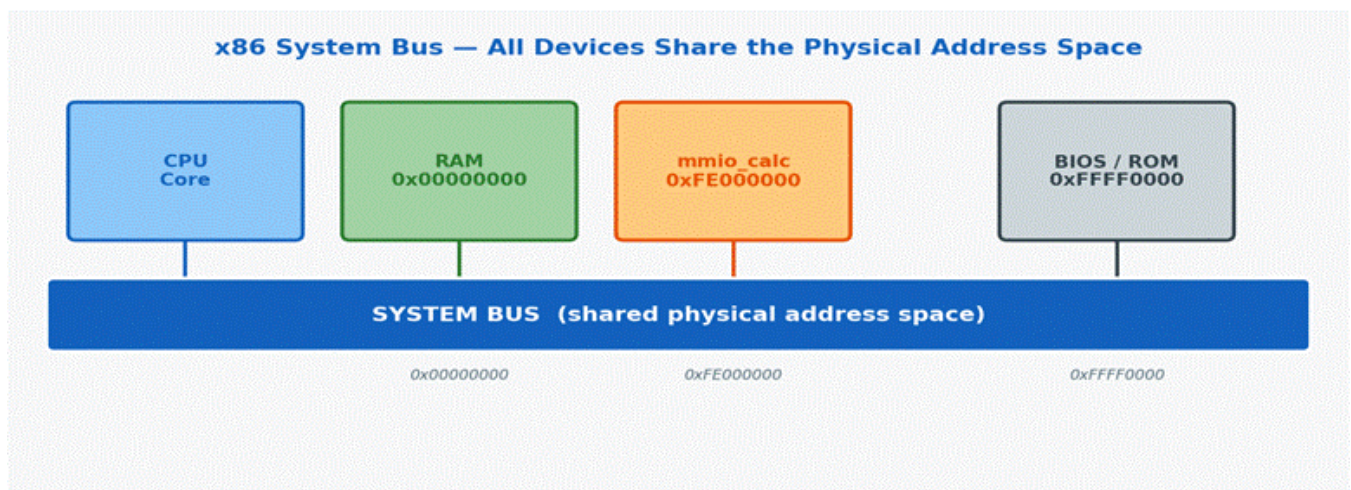
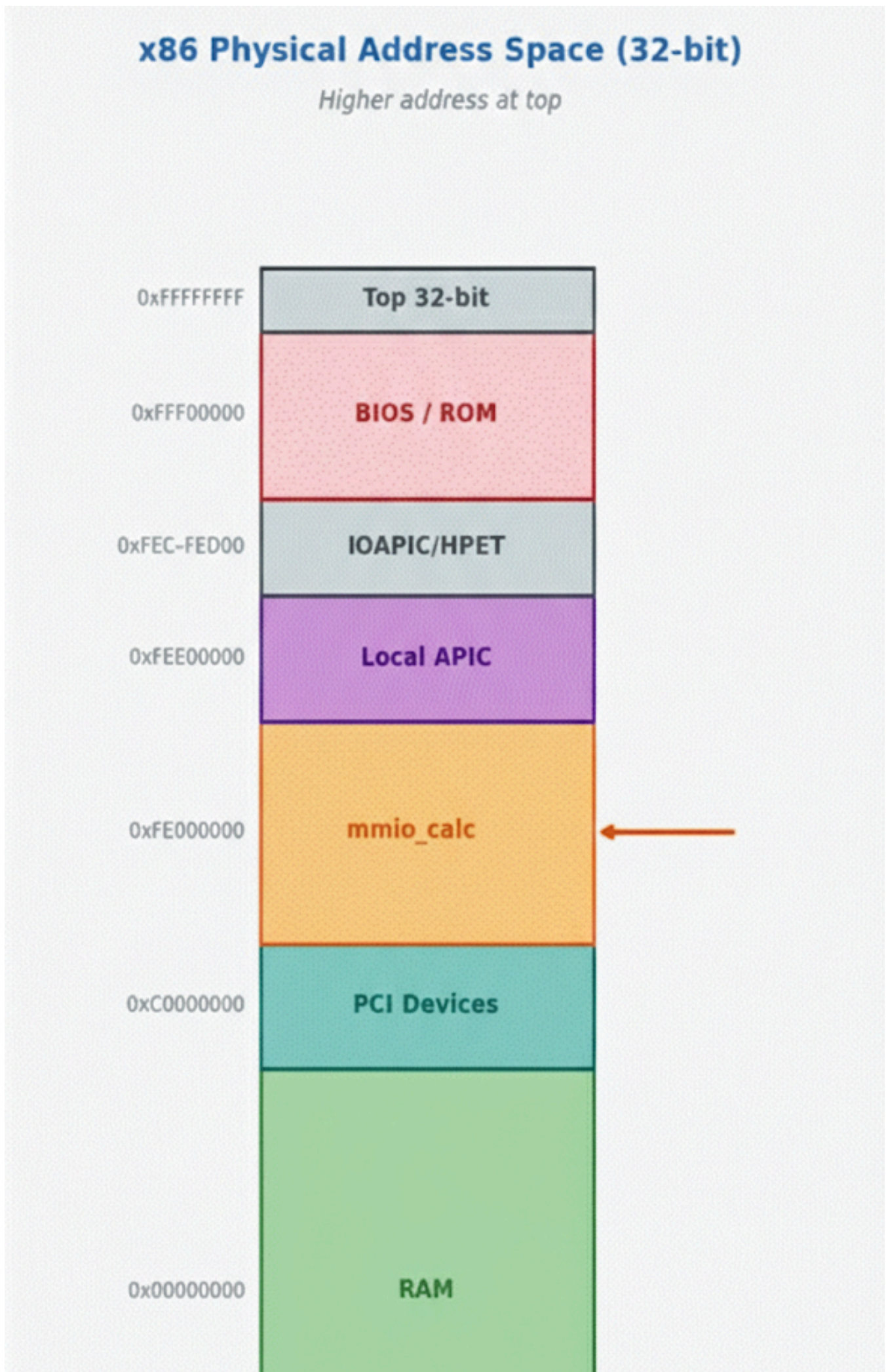


Figure 1: The x86 system bus

The address space is divided into well-defined regions:

- **RAM** starts at 0x00000000 and covers most of the low address range.
- **MMIO devices** are placed at addresses above RAM — in the 32-bit PCI hole between 3 GB and 4 GB (0xC0000000–0xFFFFFFFF).
- **ROM / BIOS** sits near the top of the 32-bit address space (around 0xFFFF0000), where the CPU begins executing after reset.

When the CPU writes to 0xFE000000, it does not hit RAM. The bus decoder routes the transaction to the device at that address. From the CPU's instruction set perspective, writing to a device register and writing to RAM are identical instructions — the difference is entirely in physical address routing, invisible to the CPU core itself.



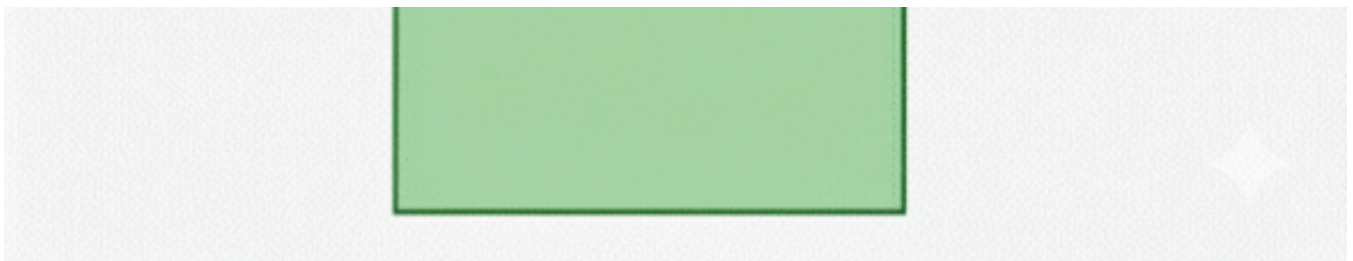


Figure 2: The x86 32-bit physical address space

MMIO Is Everywhere: Real-World Examples

MMIO is the dominant communication mechanism between software and hardware across virtually every architecture and platform. Here are concrete examples from systems you have likely already worked with.

1. x86 / PC Platform

The Local APIC (Advanced Programmable Interrupt Controller) is a per-CPU hardware unit built directly into every x86 processor since the Pentium. Each CPU core has its own dedicated APIC, responsible for receiving, prioritizing, and delivering interrupts to that specific core. It replaced the older single-chip 8259A PIC, which could only serve one CPU. The Local APIC sits at fixed physical address 0xFEE00000 and exposes roughly 64 registers, each 16 bytes apart (only the lowest 4 bytes of each slot are used). The OS interacts with it entirely through MMIO reads and writes — no special instructions needed.

Linux kernel: arch/x86/include/asm/apicdef.h

```
#define APIC_DEFAULT_PHYS_BASE 0xf0000000
```

2. ARM PL011 UART

The PL011 is ARM's standard serial port peripheral, found in QEMU's -M virt machine, the Raspberry Pi, BeagleBone, and the vast majority of ARM SoCs. It is one of the most widely encountered MMIO devices in embedded Linux development.

The device communicates entirely through a small set of memory-mapped registers. The two most important ones are the Data Register (DR) and the Flag Register (FR). To transmit a byte, the driver first reads the FR to check whether the transmit FIFO is full — if it is, the driver waits. Once FR indicates space is available, the driver writes

the byte to DR and the hardware handles the actual serial transmission. Receiving works in reverse: the driver polls FR to check whether incoming data is waiting, then reads it from DR.

Offset	Register	Description
0x00	DR	Data register (TX write / RX read)
0x18	FR	Flag register — TXFF (TX full), RXFE (RX empty)
0x24	IBRD	Integer baud rate divisor
0x2C	LCR_H	Line control — word length, parity, stop bits
0x30	CR	Control — UARTEN, TXE, RXE enable bits

Table 1: UART registers

3. STM32 GPIO — Cortex-M Bare-Metal

Every GPIO pin on an STM32 microcontroller is configured and driven through MMIO. To toggle a pin you write a bitmask to the BSRR register — no OS, no driver model, just a volatile pointer to the peripheral base address.

```
#define GPIOA_BASE    0x40020000UL
#define GPIOA_BSRR    (*(volatile uint32_t *)(GPIOA_BASE + 0x18))

GPIOA_BSRR = (1 << 5);    // set PA5 high
GPIOA_BSRR = (1 << 21);   // set PA5 low (bit 16 + pin number)
```

4. NVMe — PCIe MMIO Doorbells

NVMe SSDs expose control registers (including queue doorbells) through a PCIe Base Address Register (BAR), which the OS maps into its address space using memory-mapped I/O (MMIO). To submit commands, the host places commands in a **submission queue in host memory** and then performs a **32-bit MMIO write** to the corresponding *submission queue tail doorbell* register to notify the controller that new commands are ready for processing. The controller then fetches the commands via DMA.

The Common Pattern

Every example above follows the same protocol: Write config registers → Trigger action → Poll status → Read result

Whether it is a UART, an NVMe SSD, a GPIO controller, or an interrupt controller — the mechanism is the same. Understanding MMIO on a simple device gives you the mental model to understand all of them.

How QEMU Models the System Bus

QEMU implements the same system bus model in software. Every guest memory access is dispatched through the MemoryRegion infrastructure. Each registered region covers a physical address range and provides read/write callbacks.

When guest software accesses a registered address, QEMU intercepts it and calls your C function instead of touching emulated RAM. From the guest kernel's perspective — from the device driver's perspective — it is indistinguishable from a real hardware register write.

The key insight: a QEMU device is just a C struct with two function pointers. The entire hardware behavior lives in those callbacks. The infrastructure around it — QOM type system, MemoryRegion dispatch, bus modeling — is what makes it transparent to the guest.

A Toy Example: Building an MMIO Calculator in QEMU

1. The Device Register Map

Our toy device "mmio_calc" exposes six 32-bit registers at base address 0xFE000000, spanning a 4 KB region. The registers are laid out sequentially, each 4 bytes wide and aligned to a 4-byte boundary. Three of them are write-only — the CPU configures the operation through them. Two are read-only — the CPU retrieves the result and checks progress through them. One acts as a trigger: writing the value 1 to it fires the calculation.

This separation between configuration registers, a trigger register, and status/result registers is a deliberate and common pattern in real hardware design. It ensures the device never acts on a partially written state — the operands and operation are all committed before the trigger is touched.

Offset	Register	Access	Description
0x00	A	Write	Operand A
0x04	B	Write	Operand B
0x08	OP	Write	Operation: 0=add 1=subtract 2=multiply 3=divide
0x0C	CTRL	Write	Write 1 to trigger the calculation
0x10	RESULT	Read	Result (valid when STATUS = 2)
0x14	STATUS	Read	0=idle 1=busy 2=done 3=error

Table 2: Our calculator registers

2. The QEMU Device: "mmio_calc.c"

In QEMU's source tree, devices that don't belong to a specific bus (PCI, USB, ISA) live under `hw/misc/`. This is the right home for our calculator: a simple system bus device that maps directly into the physical address space. The file `hw/misc/mmio_calc.c` contains the entire device implementation in roughly 93 lines of C.

The device is built from four components that every QEMU device requires:

2.1 Device State

```
typedef struct MMIOCalcState {
    SysBusDevice parent_obj; // QOM base type for bus-less devices
    uint32_t a, b, op;
    uint32_t result;
    uint32_t status;
    MemoryRegion mmio; // the 4 KB mapped region
} MMIOCalcState;
```

`SysBusDevice` is QEMU's object model type for devices that map directly into the system bus address space without a PCI or ISA bus in between. The `MemoryRegion` is the handle QEMU uses to route guest memory accesses to our callbacks.

2.2 Read and Write Callbacks

```
static uint64_t calc_read(void *opaque, hwaddr addr, unsigned size) {
    MMIOCalcState *s = opaque;
    switch (addr) {
        case 0x10: return s->result;
```



```

        case 0x14: return s->status;
        default:   return 0;
    }
}

static void calc_write(void *opaque, hwaddr addr, uint64_t val, unsigned
    MMIOCalcState *s = opaque;
    switch (addr) {
    case 0x00: s->a = val; break;
    case 0x04: s->b = val; break;
    case 0x08: s->op = val; break;
    case 0x0C: if (val == 1) calc_exec(s); break;
    }
}

```

Reads to A, B, OP, and CTRL return 0 — those are write-only registers, matching how real hardware often works. Only RESULT and STATUS are readable.

2.3 Execution and Error Handling

```

static void calc_exec(MMIOCalcState *s) {
    s->status = 1;    // busy
    switch (s->op) {
    case 0: s->result = s->a + s->b; break;
    case 1: s->result = s->a - s->b; break;
    case 2: s->result = s->a * s->b; break;
    case 3:
        if (s->b == 0) { s->status = 3; return; } // divide-by-zero
        s->result = s->a / s->b;
        break;
    default: s->status = 3; return; // unknown operation
    }
    s->status = 2;    // done
}

```

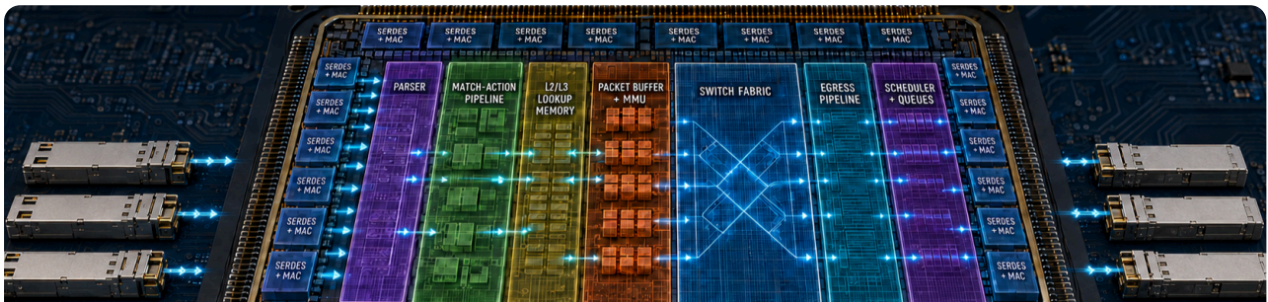
Division by zero sets STATUS=3 (error) instead of crashing QEMU. The status transitions — idle → busy → done/error — mirror how real asynchronous peripherals signal completion to the CPU.

Recommended by LinkedIn



UP Announces New Development Kit Ecosystem Tailored to...

UP! Bridge the gap · 11 months ago



Why the Network Switch ASIC Is an Engineering Marvel

Ravichandran Paramasivam · 2 weeks ago



RISC-V in Personal Computing: Promise, Reality, and a...

Ayman Alheraki · 4 months ago

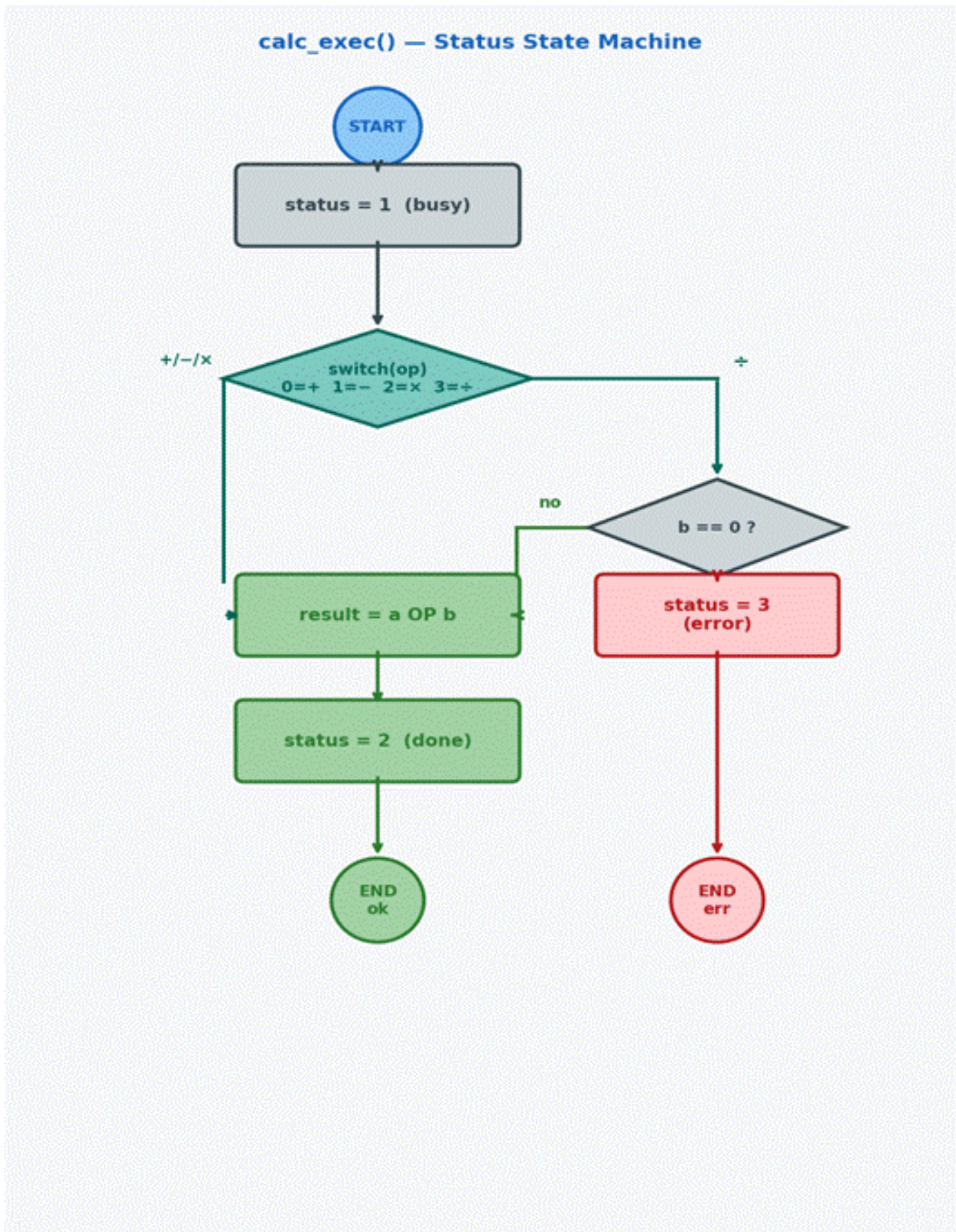


Figure 3: The calc_exec() state machine

2.4 Device Registration

```
static const MemoryRegionOps calc_ops = {
    .read      = calc_read,
    .write     = calc_write,
    .endianness = DEVICE_NATIVE_ENDIAN,
};

static void mmio_calc_init(Object *obj) {
    MMIOCalcState *s = MMIO_CALC(obj);
    memory_region_init_io(&s->mmio, obj, &calc_ops, s, "mmio-calc", 0x100);
    sysbus_init_mmio(SYS_BUS_DEVICE(obj), &s->mmio);
}

type_init(mmio_calc_register_types)
```

`type_init` is a constructor attribute — QEMU calls it at startup to register the device type by name. After registration, the device can be instantiated from anywhere in the codebase using that name.

Wiring Into the i440FX Machine: `pc_piix.c`

Having a device registered is not enough — it must be instantiated and mapped at a specific physical address. This happens inside `pc_init1()` in `hw/i386/pc_piix.c`, the function that initializes the entire Standard PC (i440FX + PIIX, 1996) machine:

```
DeviceState *mmio_calc_dev = qdev_new("mmio-calc");
sysbus_realize_and_unref(SYS_BUS_DEVICE(mmio_calc_dev), &error_fatal);
sysbus_mmio_map_overlap(SYS_BUS_DEVICE(mmio_calc_dev), 0, 0xFE000000, 1);
```

Three things happen here:

- `qdev_new("mmio-calc")` instantiates the device using the name registered in `mmio_calc.c`.

- `sysbus_realize_and_unref` calls the device's init function, which allocates and initializes the `MemoryRegion`.
- `sysbus_mmio_map_overlap(..., 0xFE000000, 1)` maps the region at physical address `0xFE000000` with priority 1, so it takes precedence over any overlapping mappings.

The address `0xFE000000` sits in the 32-bit PCI hole — above RAM, below 4 GB — exactly where real device BARs and firmware sit. It will not conflict with guest RAM.

Adding the device into QEMU meson build

Add one line to `hw/misc/meson.build` to compile the device into all system builds in QEMU:

```
system_ss.add(files('mmio_calc.c'))
```

Putting It All Together

1. Prerequisites

To compile and build the code presented in this article, the following tools must be installed on your host machine

Component	Version
OS	Ubuntu 22.04 LTS (x86_64)
GCC	11.4.0
GNU Binutils	2.38
GNU Make	4.3
Python	3.10.12
Ninja	1.10.1

Table 3: Tool chains & OS specs.

Item	Value
Version	v10.2.0
Source	gitlab.com/qemu-project/qemu
Target	x86_64-softmmu
Build system	Meson + Ninja
Machine type	pc-i440fx (Standard PC, i440FX + PIIX, 1996)

Table 4: QEMU specs.

Component	Version / Detail
Linux kernel	v6.6 LTS — make defconfig for x86_64
BusyBox	Latest master — CONFIG_STATIC=y
calc_test	gcc -static — no runtime dependencies
Initramfs	cpio -H newc gzip

Table 5: Linux and Busybox specs.

```
sudo apt install -y git build-essential ninja-build pkg-config \
    libgl2.0-dev libpixman-1-dev python3 python3-venv \
    libfdt-dev zlib1g-dev
```

2. Building QEMU

```
git clone https://gitlab.com/qemu-project/qemu.git
cd qemu && git checkout v10.2.0
./configure --target-list=x86_64-softmmu
make -j$(nproc)
```

3. The Full Stack: Kernel + BusyBox + Userspace

To run code that exercises the device we need a complete but minimal system:

- Linux kernel v6.6 — provides /dev/mem and the serial console
- BusyBox — a single statically-linked binary providing ~300 Unix utilities

- `calc_test` — the test program, statically compiled against `glibc`
- `initramfs` — a gzip-compressed `cpio` archive loaded directly into RAM at boot

3.1 Build the Linux Kernel

```
sudo apt install -y git build-essential libncurses-dev bison flex \
    libssl-dev bc zlib1g-dev libelf-dev
```

```
git clone https://github.com/torvalds/linux.git
cd linux && git checkout v6.6
make ARCH=x86_64 defconfig
make -j$(nproc) ARCH=x86_64
# Output: arch/x86/boot/bzImage
```

`make defconfig` produces a minimal but functional `x86_64` config with `/dev/mem` support and serial console enabled. No bootloader is needed — QEMU loads the kernel directly via its built-in boot stub.

3.2 Build BusyBox (Static)

```
git clone https://git.busybox.net/busybox
cd busybox && make defconfig
echo 'CONFIG_STATIC=y' >> .config
make olddefconfig
make -j$(nproc)
make CONFIG_PREFIX=../rootfs install
```

The critical requirement for an `initramfs` is static linking — no shared libraries are available at boot time. `make install` populates `rootfs/` with `bin/`, `sbin/`, `usr/bin/`, and `usr/sbin/` — all symlinks pointing to a single busybox binary.

3.3 Build and Install the Test Program

```
gcc -static -o calc_test calc_test.c
cp calc_test rootfs/bin/
```

3.4 Pack the Root Filesystem

```
cd rootfs
mkdir -p proc sys dev tmp

cat > init << 'EOF'
#!/bin/sh
mount -t proc      proc      /proc
mount -t sysfs     sysfs     /sys
mount -t devtmpfs  devtmpfs  /dev
exec /bin/sh
EOF

chmod +x init
find . | cpio -H newc -o > ../rootfs.cpio
cd .. && gzip rootfs.cpio
```

The /init script is the first userspace process (PID 1). It mounts proc, sysfs, and devtmpfs — the last of which auto-creates /dev/mem and /dev/null — then drops into the BusyBox shell.

3.5 Boot Everything in QEMU

```
./qemu/build/qemu-system-x86_64 \
  -kernel linux/arch/x86/boot/bzImage \
  -initrd rootfs.cpio.gz \
  -append "console=ttyS0 root=/dev/ram rdinit=/init" \
  -nographic
```


QEMU loads both images, boots the kernel, extracts the initramfs into RAM, runs /init, and hands us a shell. The mmio_calc device is already live at 0xFE000000 — wired in by pc_init1() before the kernel even starts.

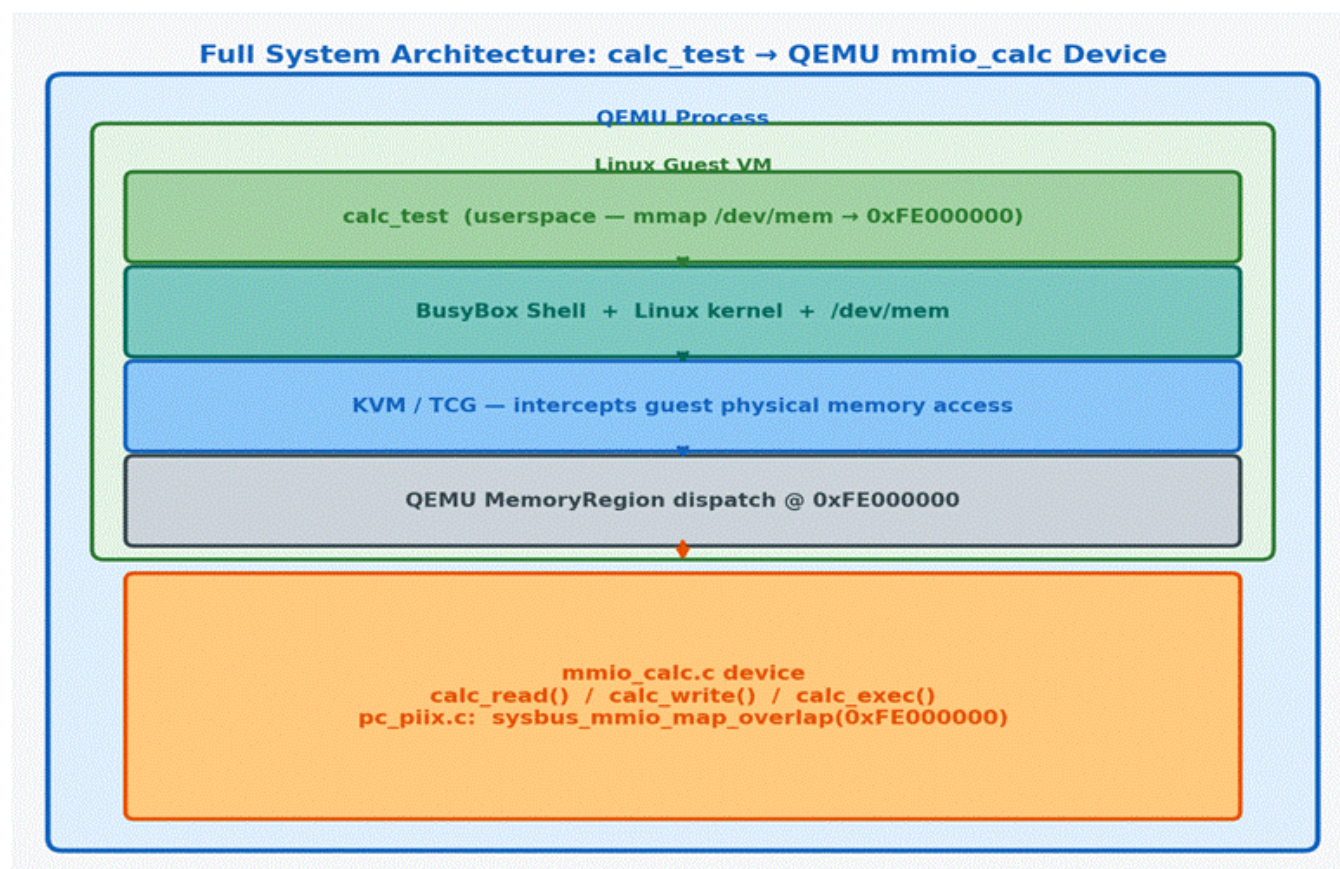


Figure 4: Full system architecture

Talking to the Device from Userspace

To test our device, we need to write a C program that opens /dev/mem and uses mmap() to map physical address 0xFE000000 into its virtual address space. After that, register access is straightforward array indexing:

```
volatile uint32_t *calc = mmap(NULL, 0x1000,
    PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0xFE000000);
```

```
calc[0] = a; // A @ 0xFE000000
```



Top Content

People

Learning

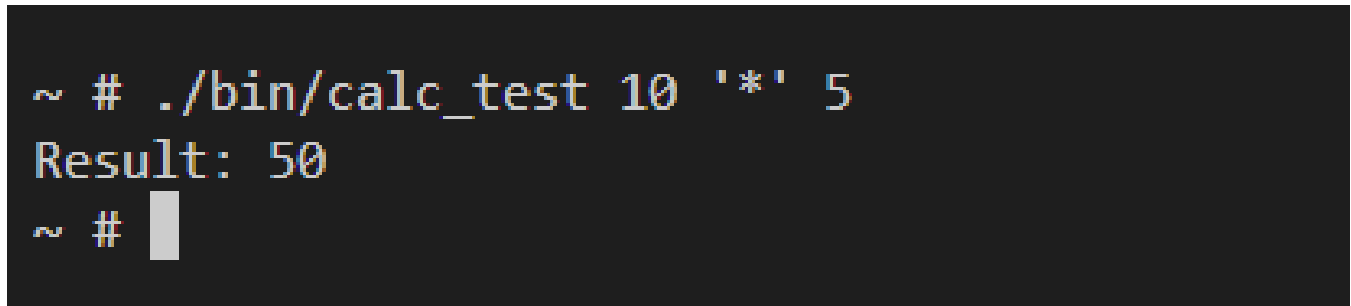
Jobs

Games

Get the app

```
printf("Result: %u\n", calc[4]);           // RESULT @ 0xFE000010
```

The program accepts command-line arguments so you can use it interactively from the BusyBox shell:



```
~ # ./bin/calc_test 10 '*' 5
Result: 50
~ #
```

Figure 6: Userspace program

Why volatile Is Not Optional?

`volatile uint32_t *calc` — this qualifier is critical. When the compiler sees `while (calc[5] != 2)`, it is allowed to reason: "this variable has no observable side effects from my perspective — I'll read it once and cache the value." Without `volatile`, on an optimized build, the poll loop may never re-read the register and spin forever regardless of what the device does.

`volatile` tells the compiler: every access through this pointer must emit a real memory load or store instruction — do not cache, do not reorder, do not eliminate. This is why every hardware register pointer in every real driver is either `volatile`, or accessed through explicit `readl()/writel()` barrier macros. The hardware changes memory behind the compiler's back — the compiler must be told.

Conclusion

Building a complete system from the ground up — device emulation inside QEMU, kernel, root filesystem, and userspace application — forces abstractions to become concrete in a way that documentation alone cannot achieve. Several fundamental principles only reveal themselves clearly once the full stack is assembled and running.

- **MMIO is a memory access, nothing more.** From the CPU's perspective, writing to a device register and writing to a RAM location are architecturally identical operations. What differentiates them is the physical address. The bus fabric — or in QEMU's case, the `MemoryRegion` dispatch layer — determines which endpoint responds. This is the central

insight that underlies every device driver in the Linux kernel, regardless of the peripheral or bus architecture.

- **QEMU's device model is well-structured and approachable.** Adding a new device requires defining a state structure, implementing two callbacks, and registering a type. The surrounding infrastructure — the QOM type system, the MemoryRegion API, the SysBus layer — handles the complexity of making that device transparent to the guest operating system. The device implementation itself remains focused and minimal. /dev/mem is a valuable bring-up tool with well-understood trade-offs. It grants userspace direct read and write access to physical addresses, bypassing the kernel driver model entirely. In production environments it is appropriately restricted via CONFIG_STRICT_DEVMEM. During hardware bring-up and driver prototyping, however, it allows a device to be exercised and validated before any kernel code is written — significantly compressing the development cycle.
- **A functional Linux environment requires less than most engineers assume.** A kernel image, a statically linked userspace, and a compressed initramfs archive are sufficient to boot an interactive system against a custom virtual device. Understanding this minimal footprint has practical value in embedded and resource-constrained contexts.
- The arithmetic performed by the calculator is beside the point. The register map, the status state machine, the MemoryRegion callbacks, the mmap-based userspace access — these are the transferable skills. The same patterns appear in NVMe queue doorbells, PCIe BAR registers, ARM SoC peripherals, and DMA engine control interfaces. The device changes; the discipline does not.



Like



Comment



Share



57

To view or add a comment, [sign in](#)

More articles by Ahmed Dajani

Jul 31, 2026

Building a PCI Device Before the Silicon Exists: A QEMU Case Study

Every hardware project eventually hits the same bottleneck: software has to wait for silicon. Drivers can't be written...

257 · 13 Comments

Others also viewed



Introduction to ARM Architecture

Tenura Pinsara · 5mo



CPU: Brain of computer

Mohamed Aasil M · 2y



Part 3: Low-Cost Voice Cloning on Pure CPU Servers (Without Retraining)



Krishankant Saraswat · 2mo



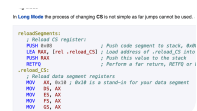
IPC Communication with Texas Instruments' IPC Mechanisms

Hammami Anis · 1y



Question about packet handling in high bandwidth large scale environments.

Mrityunjoy Chattopadhyay · 9y



x86 protected mode and Long Mode x86-64 and the equivalents on ARM.

François Piednoel de Normandie · 2y

Show more ▼

Explore content categories

Career

Productivity

Finance

Soft Skills & Emotional Intelligence

Project Management

Education

Show more ▼

© 2026

Accessibility

Privacy Policy

Copyright Policy

Guest Controls

About

User Agreement

Cookie Policy

Brand Policy

Community Guidelines

Language